

Rapport final

Studio génératif interactif - projet Flask de créativité numérique

Élément	Information
Date de vérification	26/06/2026
Période de travail	04/03/2026 au 26/06/2026
Dépôt GitHub	https://github.com/HoSSamOuammi/Digital_Thinking
Livrable	Code complet, application Flask, README et rapport final de 2-3 pages.

1. Concept et direction artistique

Le projet fonctionne comme un petit studio numérique. L'utilisateur peut partir de réglages simples, d'un CSV ou d'une image, produire un rendu, puis le retrouver dans une galerie.

La direction artistique reste calme: interface claire, cartes sobres, couleurs limitées et textes français. Le but est de montrer le fonctionnement du studio sans noyer la démonstration dans la décoration.

2. Modules implémentés

Module	Fonction	Résultat
Tableau de bord	Accueil du studio, compteurs et raccourcis vers les ateliers.	Une entrée simple pour présenter le projet rapidement.
Atelier génératif	Séries visuelles, palettes, graine, densité, taille du canevas et accents dessinés.	Visuels exportés puis visibles dans la galerie.
Données visuelles	Lecture d'un CSV ou d'un jeu de démonstration, nettoyage puis rendu graphique.	Images de visualisation prêtes à télécharger.
Outils médias	Effets image: sépia, contours, glitch, rotation, palette dominante. Audio si ffmpeg est disponible.	Fichiers transformés sans bloquer le reste de l'application.
Galerie	Liste séparée des images et audios générés, avec pagination.	Trace concrète du parcours complet: créer, retrouver, télécharger.
Équipe	Profils, rôles, emails et photos chargées depuis static/Admins.	Présentation propre du groupe dans l'application.

3. Outils utilisés et pipeline technique

Outil	Utilisation
Flask / Jinja2	Routes serveur, rendu HTML et formulaires.
Pillow	Effets image et export des fichiers traités.
Pandas / NumPy	Lecture et préparation des données numériques.
Matplotlib	Création des visualisations à partir des données.
PyDub / ffmpeg	Traitement audio lorsque la machine le permet.
unittest	Tests des routes, formulaires, exports et nettoyages.
Git / GitHub	Historique du travail et dépôt final à rendre.

Pipeline de fonctionnement:

- L'utilisateur part du tableau de bord et choisit un atelier.
- Flask reçoit le formulaire et vérifie les valeurs utiles.
- Le module Python concerné génère ou transforme le contenu.
- Le résultat est enregistré dans static/generated.
- La page affiche le fichier final et propose le téléchargement.
- Les tests rejouent les parcours importants pour vérifier que rien ne casse.

4. Challenges rencontrés et solutions

Challenge	Solution
Un app.py devenu trop chargé	Séparer la configuration, les routes, les formulaires, le stockage et les modules métier.
Une interface qui devait paraître terminée	Reprendre les textes en français et choisir un style plus sobre, plus proche d'un outil de travail.
L'audio dépend de ffmpeg	Garder le module audio optionnel pour que l'application fonctionne même si ffmpeg n'est pas installé.
Les fichiers générés peuvent vite s'accumuler	Limiter les caches, nettoyer les imports temporaires et paginer la galerie.

5. Équipe et livrables vérifiés

Photo	Membre	Rôle
	Aya EL Amrani	Structure Flask, configuration, formulaires, stockage et routes.
	Khadija Baskar	Textes français, libellés, cohérence des intitulés et contenu des pages.
	Hossam OUammi	Intégration Flask, interface, médias, galerie et pages de présentation.
	Abderrahmane El Garti	Tests fonctionnels, documentation, rapport et analyse technique.

Livrable	Vérification
Base de code complète	Dépôt avec app.py, studio, modules, templates, static, tests et requirements.txt.
Application Flask	Lancement par python app.py; les pages principales répondent correctement.
README	Installation, lancement, tests, structure et livrables finaux.
Rapport final	PDF de 2-3 pages couvrant le concept, les modules, le pipeline et les challenges.